



**Università
degli Studi
di Palermo**



Introduzione a NLTK

CORSO DI NATURAL LANGUAGE PROCESSING (ELABORAZIONE DEL LINGUAGGIO NATURALE)

a.a. 2026r/2027

Prof. Roberto Pirrone



Installazione e importazione dei pacchetti

NLTK — Natural Language Toolkit

Installazione

Terminale / notebook

```
# Da terminale (pip)
pip install nltk
pip install --upgrade nltk      # aggiornamento

# Con conda
conda install -c conda-forge nltk

# In un notebook Jupyter / Colab
!pip install nltk
```

Importazione

Python

```
import nltk
print(nltk.__version__)      # es. 3.10.3

# Import selettivi dei moduli più usati
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.corpus import stopwords, wordnet
from nltk import pos_tag, ne_chunk, FreqDist
```

Da sapere

Il pacchetto contiene il codice; i dati linguistici (modelli, corpora, lessici) NON sono inclusi e vanno scaricati a parte (slide 3).

Moduli principali

Modulo	Contenuto
nltk.tokenize	tokenizzatori di frasi e di parole
nltk.stem	stemmer e lemmatizzatore WordNet
nltk.tag	POS tagger (perceptron, n-gram, HMM...)
nltk.chunk	chunking e NER (ne_chunk, RegexpParser)
nltk.corpus	corpora, lessici, WordNet, FrameNet
nltk.lm	language model a n-grammi (MLE, KN...)
nltk.probability	FreqDist, ConditionalFreqDist
nltk.util	ngrams, bigrams, everygrams
nltk.metrics	edit_distance, jaccard_distance
nltk.collocations	ricerca di collocazioni
nltk.sentiment	VADER e analisi del sentimento
nltk.text	classe Text (concordance, similar...)

Esempio minimo

Python

```
import nltk
tokens = nltk.word_tokenize('NLTK makes NLP easy.')
print(nltk.pos_tag(tokens))
```

Caricamento delle risorse linguistiche

nltk.download() · nltk.data · corpora e lessici

Scaricare le risorse

Python

```
import nltk

nltk.download('punkt_tab')      # tokenizzatore frasi/parole
nltk.download('stopwords')      # stopwords multilingua
nltk.download('wordnet')        # lessico WordNet
nltk.download('omw-1.4')        # WordNet multilingua
nltk.download('averaged_perceptron_tagger_eng') # POS
nltk.download('maxent_ne_chunker_tab') # NER
nltk.download('words')          # elenco parole inglesi
nltk.download('vader_lexicon')  # sentiment
nltk.download('framenet_v17')   # FrameNet 1.7
nltk.download('wordnet_ic')     # information content
```

Altre modalità

```
nltk.download()                # gestore grafico
nltk.download('popular')        # pacchetti più comuni
nltk.download('all')            # tutto: molto pesante
nltk.download('wordnet', download_dir='./nltk_data')
```

Attenzione ai nomi

Dalla 3.9 i modelli 'punkt', 'averaged_perceptron_tagger' e 'maxent_ne_chunker' sono sostituiti da 'punkt_tab', '..._eng' e '..._tab'. Se compare LookupError, scaricare la risorsa citata nel messaggio d'errore.

Dove vengono cercati i dati

Python

```
print(nltk.data.path) # cartelle di ricerca
# di solito ~/nltk_data, /usr/share/nltk_data ...

nltk.data.path.append('/mio/percorso/nltk_data')
# oppure variabile d'ambiente: NLTK_DATA

nltk.data.find('tokenizers/punkt_tab') # verifica
```

Uso di corpora e lessici

Python

```
from nltk.corpus import stopwords, gutenberg

sw = stopwords.words('italian') # lista stopwords
print(stopwords.fileids())      # lingue disponibili

print(gutenberg.fileids()[:3])  # testi del corpus
emma = gutenberg.words('austen-emma.txt')
```

Lingue diverse dall'inglese

Per l'italiano NLTK offre stopwords, Snowball stemmer e Punkt.
Per POS, NER e lemmi in italiano: spaCy, Stanza, simplemma.

Tokenizzazione: frasi e parole

sent_tokenize · word_tokenize

sent_tokenize — segmentazione in frasi

Python

```
from nltk.tokenize import sent_tokenize, word_tokenize

testo = ("NLTK è una libreria per l'NLP. "
        "Funziona con molte lingue! La usi già?")
frasi = sent_tokenize(testo, language='italian')
print(frasi)
```

Output

```
["NLTK è una libreria per l'NLP.",
 'Funziona con molte lingue!',
 'La usi già?']
```

word_tokenize — segmentazione in parole

Python

```
print(word_tokenize('Non è un problema, vero?',
                    language='italian'))
```

Output

```
['Non', 'è', 'un', 'problema', ',', 'vero', '?']
```

Come funziona

word_tokenize = sent_tokenize (modello Punkt) + tokenizer
in stile Penn Treebank applicato a ogni frase.

Confronto con str.split()

Python

```
s = "Dr. Smith's e-mail costs $9.99, isn't it?"
print(s.split())
print(word_tokenize(s))
```

Output

```
['Dr.', "Smith's", 'e-mail', 'costs', '$9.99,',
 'isn't', 'it?']
['Dr.', 'Smith', "'s", 'e-mail', 'costs', '$',
 '9.99', ',', 'is', "n't", 'it', '?']
```

Limiti con l'italiano

I modelli sono tarati sull'inglese: con l'italiano l'apostrofo non viene separato ("L'esame", "nell'aula" restano interi). Per separarlo si può usare un RegexpTokenizer (slide 5).

```
print(word_tokenize("L'esame è difficile, ma c'è tempo."))
```

Output

```
["L'esame", 'è', 'difficile', ',', 'ma', "c'è", 'tempo', '.']
```

Tipologie di tokenizzazione

tokenizzatori a regole, per social media, per espressioni multi-parola

Panoramica dei tokenizzatori

Classe / funzione	Caratteristiche
word_tokenize	Treebank + Punkt (richiede punkt_tab)
WhitespaceTokenizer	divide sugli spazi bianchi
wordpunct_tokenize	sequenze \w+ e punteggiatura separate
RegexpTokenizer	token (o separatori) definiti da regex
TreebankWordTokenizer	regole Penn Treebank, senza sent. split
TweetTokenizer	hashtag, @menzioni, emoticon, URL
MWETokenizer	fonde espressioni multi-parola
PunktSentenceTokenizer	segmentatore di frasi addestrabile
BlanklineTokenizer	divide in paragrafi (righe vuote)

span_tokenize — posizioni dei token

Python

```
tk = RegexpTokenizer(r'\w+')
print(list(tk.span_tokenize('ciao mondo bello')))
```

Output

```
[(0, 4), (5, 10), (11, 16)]
```

Confronto su una stessa stringa

Python

```
t = "Dr. Smith's e-mail is smith@unipa.it; it costs $9.99"
WhitespaceTokenizer().tokenize(t)
wordpunct_tokenize(t)
RegexpTokenizer(r'\w+').tokenize(t)
```

Output

```
['Dr.', "Smith's", 'e-mail', 'is', 'smith@unipa.it;', ...]
['Dr', '.', 'Smith', '"', 's', 'e', '-', 'mail', 'is', ...]
['Dr', 'Smith', 's', 'e', 'mail', 'is', 'smith', ...]
```

Italiano: apostrofo, social, multi-parola

Python

```
it = RegexpTokenizer(r"^[\W\d_]+'?|^[^\w\s]")
it.tokenize("L'esame è difficile, c'è tempo.")

tw = TweetTokenizer(preserve_case=False,
                    strip_handles=True, reduce_len=True)
tw.tokenize('@user Soooo coool!!! #NLP :-)')
mwe = MWETokenizer(['machine', 'learning'], separator='_')
mwe.tokenize('I study machine learning'.split())
```

Output

```
['L', 'esame', 'è', 'difficile', ',', 'c', 'è', 'tempo', '.']
['sooo', 'coool', '!', '!', '!', '#nlp', ':-)']
['I', 'study', 'machine_learning']
```

Stemming

riduzione delle parole alla radice tramite regole sui suffissi

Gli stemmer di NLTK

Python

```
from nltk.stem import (PorterStemmer, LancasterStemmer,
                       SnowballStemmer, RegexpStemmer)

porter = PorterStemmer()
lancaster = LancasterStemmer()          # più aggressivo
snow_en = SnowballStemmer('english')
snow_it = SnowballStemmer('italian')

porter.stem('studies')                  # 'studi'
snow_it.stem('mangiando')               # 'mang'
RegexpStemmer('ing$|s$|ed$', min=4).stem('walking')
```

Lingue supportate da Snowball

```
print(SnowballStemmer.languages)
```

Output

```
('arabic', 'danish', 'dutch', 'english', 'finnish', 'french',
 'german', 'hungarian', 'italian', 'norwegian', 'porter',
 'portuguese', 'romanian', 'russian', 'spanish', 'swedish')
```

Inglese: tre stemmer a confronto

Parola	Porter	Lancaster	Snowball
running	run	run	run
flies	fli	fli	fli
happily	happili	happy	happili
studies	studi	study	studi
better	better	bet	better
generously	gener	gen	generous

Italiano: SnowballStemmer('italian')

Parola	Radice	Parola	Radice
mangiando	mang	corsa	cors
mangiare	mang	correre	corr
mangiato	mang	università	univers
bellissimo	bellissim	universale	universal

Cosa aspettarsi

Euristico: la radice può non essere una parola valida (studi),
unire parole non correlate (over-stemming: 'better' -> 'bet')
o non unire forme dello stesso lemma (under-stemming).

Lemmatizzazione

WordNetLemmatizer — dalla forma flessa al lemma del dizionario

WordNetLemmatizer

Python

```
from nltk.stem import WordNetLemmatizer
lem = WordNetLemmatizer() # richiede wordnet

lem.lemmatize('geese')      # 'goose' (pos='n')
lem.lemmatize('running')    # 'running' (default n)
lem.lemmatize('running', pos='v') # 'run'
lem.lemmatize('better', pos='a') # 'good'
lem.lemmatize('was', pos='v') # 'be'
```

Parte del discorso

Il parametro pos vale 'n' (nome, default), 'v' (verbo), 'a' (aggettivo), 'r' (avverbio). Senza il POS corretto il lemma può restare invariato.

Stemming vs lemmatizzazione

Parola	Stem (Porter)	Lemma
studies	studi	study
geese	gees	goose
was	wa	be (pos='v')
better	better	good (pos='a')
corpora	corpora	corpus

Lemmatizzare una frase (con POS tagging)

Python

```
from nltk import pos_tag, word_tokenize
from nltk.corpus import wordnet

def wn_pos(tag): # tag Penn -> tag WordNet
    return {'J': wordnet.ADJ, 'V': wordnet.VERB,
            'R': wordnet.ADV}.get(tag[0], wordnet.NOUN)

s = 'The children were studying old methods'
lemmi = [lem.lemmatize(w, wn_pos(t))
          for w, t in pos_tag(word_tokenize(s))]
print(lemmi)
```

Output (indicativo)

```
['The', 'child', 'be', 'study', 'old', 'method']
```

E per l'italiano?

WordNetLemmatizer lavora solo sull'inglese. Per l'italiano: spaCy (it_core_news_sm), Stanza o simplemma. WordNet multilingua (omw-1.4) dà sinonimi, non lemmatizza: wordnet.synsets('cane', lang='ita')

POS tagging

assegnazione della categoria grammaticale a ogni token

pos_tag — tagger pre-addestrato

Python

```
from nltk import pos_tag, word_tokenize

tok = word_tokenize('The quick brown fox jumps over the lazy dog')
print(pos_tag(tok))                # tagset Penn Treebank
print(pos_tag(tok, tagset='universal')) # tagset universale
```

Output (indicativo)

```
[('The', 'DT'), ('quick', 'JJ'), ('brown', 'JJ'), ('fox', 'NN'),
 ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'),
 ('dog', 'NN')]
[('The', 'DET'), ('quick', 'ADJ'), ('brown', 'ADJ'), ('fox', 'NOUN'),
 ('jumps', 'VERB'), ('over', 'ADP'), ('the', 'DET'), ('lazy', 'ADJ'),
 ('dog', 'NOUN')]
```

Tagger addestrabili

```
from nltk.corpus import treebank
from nltk.tag import UnigramTagger, DefaultTagger
sents = treebank.tagged_sents() # download: treebank
tg = UnigramTagger(sents[:3000], backoff=DefaultTagger('NN'))
print(tg.accuracy(sents[3000:]))
```

Tag principali (Penn Treebank)

Tag	Significato	Tag	Significato
NN	nome singolare	JJ	aggettivo
NNS	nome plurale	RB	avverbio
NNP	nome proprio	DT	determinante
VB	verbo (base)	IN	preposizione
VBD	verbo passato	PRP	pronome personale
VBG	gerundio / part. pres.	CC	cong. coordinante
VBN	participio passato	CD	numero cardinale
VBZ	3a pers. sing. pres.	MD	verbo modale

```
nltk.help.upenn_tagset('VB.*') # descrizione dei tag
```

Lingue

pos_tag supporta lang='eng' (default) e 'rus'. Non esiste un tagger italiano integrato: per l'italiano usare spaCy, Stanza o addestrare un tagger su un corpus annotato (Universal Dependencies).



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Named Entity Recognition (NER)

ne_chunk · alberi di chunk · estrazione delle entità

ne_chunk sul testo con POS tag

Python

```
from nltk import word_tokenize, pos_tag, ne_chunk

frase = 'Barack Obama was born in Hawaii.'
tagged = pos_tag(word_tokenize(frase))
tree = ne_chunk(tagged)
print(tree)
```

Output (indicativo: dipende dal modello)

```
(S (PERSON Barack/NNP) (PERSON Obama/NNP)
  was/VBD born/VBN in/IN (GPE Hawaii/NNP) ./.)
```

Estrarre le entità dall'albero

Python

```
for nodo in tree:
    if hasattr(nodo, 'label'):          # sotto-albero = entità
        testo = ' '.join(w for w, t in nodo.leaves())
        print(nodo.label(), '->', testo)

ne_chunk(tagged, binary=True)         # solo etichetta 'NE'
```

Etichette e formati

Etichetta	Significato
PERSON	persone
ORGANIZATION	organizzazioni
GPE	entità geo-politiche (città, stati)
LOCATION	luoghi non geo-politici
FACILITY	strutture, edifici

IOB e RegexpParser

```
from nltk.chunk import tree2conlltags
print(tree2conlltags(tree))  # (parola, POS, IOB)

# Chunking su misura con una grammatica
from nltk import RegexpParser
cp = RegexpParser('NP: {<DT>?<JJ>*<NN.*>+}')
print(cp.parse(tagged))
```

Limiti

ne_chunk usa un modello addestrato su testo inglese: per l'italiano non è affidabile. Alternative: spaCy, Stanza o pipeline 'ner' di Hugging Face Transformers.

Funzioni di utilità (1)

frequenze, n-grammi, concordanze, collocazioni

FreqDist — distribuzione di frequenza

Python

```
from nltk import FreqDist
toks = 'il gatto mangia il topo e il gatto dorme'.split()
fd = FreqDist(toks)
print(fd.most_common(3))
print(fd['gatto'], fd.N(), fd.B(), fd.freq('il'))
print(fd.hapaxes())
fd.plot(30)          # grafico   |   fd.tabulate()
```

Output

```
[('il', 3), ('gatto', 2), ('mangia', 1)]
2 9 6 0.3333333333333333
['mangia', 'topo', 'e', 'dorme']
```

N-grammi

```
from nltk.util import bigrams, trigrams, ngrams, everygrams
list(bigrams(toks))[:3]
list(ngrams(toks, 4))[:1]
list(everygrams(['a','b','c'], max_len=2))
```

Output

```
[('il','gatto'), ('gatto','mangia'), ('mangia','il')]
[('il', 'gatto', 'mangia', 'il')]
[('a',), ('a','b'), ('b',), ('b','c'), ('c',)]
```

La classe Text

Python

```
from nltk import Text
from nltk.corpus import gutenbergt = Text(gutenberg.words('austen-emma.txt')) # o Text(tokens)
t.concordance('marriage', width=60, lines=3) # contesti
t.similar('woman') # parole in contesti simili
t.common_contexts(['man', 'woman'])
t.collocations() # coppie che co-occorrono
t.dispersion_plot(['Emma', 'Knightley'])
```

Collocazioni e frequenze condizionate

Python

```
from nltk.collocations import (BigramCollocationFinder,
                               BigramAssocMeasures)
fin = BigramCollocationFinder.from_words(tokens)
fin.apply_freq_filter(3)
fin.nbest(BigramAssocMeasures().pmi, 10)

from nltk import ConditionalFreqDist
cfd = ConditionalFreqDist((genere, parola) for ...)
cfd['news'].most_common(5)
```

Funzioni di utilità (2)

stopword, distanze tra stringhe, sentiment, metriche di valutazione

Pulizia del testo e stopwords

Python

```
import string
from nltk.corpus import stopwords

sw = set(stopwords.words('italian')) | set(string.punctuation)
puliti = [t.lower() for t in tokens
          if t.lower() not in sw and t.isalpha()]
```

Distanze tra stringhe e insiemi

Python

```
from nltk import edit_distance
from nltk.metrics import jaccard_distance

edit_distance('kitten', 'sitting')          # 3
edit_distance('intention', 'execution')      # 5
jaccard_distance(set('night'), set('nacht')) # 0.5714
```

Distanza di Levenshtein

edit_distance conta inserzioni, cancellazioni e sostituzioni;
con transpositions=True considera anche gli scambi adiacenti.

Analisi del sentimento (VADER)

Python

```
from nltk.sentiment import SentimentIntensityAnalyzer
sia = SentimentIntensityAnalyzer() # vader_lexicon
sia.polarity_scores('NLTK is great!')
# dict: neg, neu, pos, compound (solo inglese, testi brevi)
```

Valutazione: accuracy e matrice di confusione

Python

```
from nltk.metrics import accuracy, ConfusionMatrix
ref = 'DET NN VB DET JJ NN NN IN DET NN'.split()
test = 'DET VB VB DET NN NN NN IN DET NN'.split()
accuracy(ref, test) # 0.8
print(ConfusionMatrix(ref, test))
# anche: precision, recall, f_measure (su insiemi)
```

Output

```
      | D      |
      | E I J N V |
      | T N J N B |
-----+-----+
DET | <3> . . . |
IN  | .<1> . . . |
JJ  | . .<.>1 . |
NN  | . . .<3>1 |
VB  | . . . .<1>|
-----+-----+
(row = reference; col = test)
```

Language modeling con n-grammi (1)

nltk.lm — stimare $P(\text{parola} \mid \text{contesto})$ dai conteggi

Modello a N-grammi

Concetti

Assunzione di Markov: la parola dipende solo dalle N-1 precedenti

Stima MLE (bigrammi): $P(w_i \mid w_{i-1}) = C(w_{i-1}, w_i) / C(w_{i-1})$

Frase: prodotto delle $P(w_i \mid \text{contesto})$, di solito in scala log

Perplexity: $PP = P(w_1 \dots w_n)$ elevato a $(-1/n)$; più bassa = meglio

Smoothing: evita $P = 0$ per n-grammi mai visti nel training

Preparazione dei dati

Python

```
from nltk.lm.preprocessing import padded_everygram_pipeline

corpus = ["il gatto mangia il topo",
          "il gatto dorme sul divano",
          "il cane mangia la carne", "il topo dorme",
          "il cane dorme sul divano", "il gatto vede il cane"]
sents = [s.split() for s in corpus] # già tokenizzate

n = 2 # ordine del modello
train, vocab = padded_everygram_pipeline(n, sents)
```

Attenzione

`padded_everygram_pipeline` aggiunge `<s>` e `</s>` a ogni frase e restituisce due generatori: n-grammi di ordine 1..N e parole per il vocabolario. Si consumano una sola volta: vanno rigenerati.

Addestramento e interrogazione

Python

```
from nltk.lm import MLE
lm = MLE(n) # modello di ordine n
lm.fit(train, vocab)

len(lm.vocab) # 14 (con <s>, </s>, <UNK>)
lm.counts.unigrams['il'] # 8
lm.counts[['il']]['gatto'] # 3
lm.score('gatto', ['il']) # 0.375 = 3/8
lm.score('pesce', ['il']) # 0.0 (mai visto)
lm.logscore('gatto', ['il']) # -1.415 (log2)
lm.vocab.lookup(['il', 'pesce']) # ('il', '<UNK>')
```

Ordine superiore: trigrammi

Python

```
train3, vocab3 = padded_everygram_pipeline(3, sents)
lm3 = MLE(3)
lm3.fit(train3, vocab3)
lm3.score('divano', ['dorme', 'sul']) # 1.0
lm3.context_counts(lm3.vocab.lookup(['dorme', 'sul']))
# FreqDist({'divano': 2})
```

Il contesto è sempre una lista di N-1 parole:
`score(parola, contesto)`.

Language modeling con n-grammi (2)

smoothing, perplexity, parole sconosciute, generazione di testo

Modelli disponibili in nltk.lm

Classe	Idea
MLE	frequenze relative, nessuno smoothing
Lidstone(gamma, n)	somma gamma a ogni conteggio
Laplace	Lidstone con gamma = 1 (add-one)
WittenBellInterpolated	interpola tra gli ordini
KneserNeyInterpolated	sconto + probabilità di continuazione
StupidBackoff	backoff con alpha (non normalizzato)

Perplexity su tre frasi di test

Modello	frase vista	bigr. nuovo	parola ignota
MLE	1.51	inf	inf
Laplace	5.27	7.94	9.30
Lidstone(0.1)	2.08	6.51	8.94
WittenBell	2.12	5.57	inf
KneserNey	1.57	6.93	inf

Come leggere la tabella

vista: 'il gatto dorme sul divano'; nuovo: 'il topo mangia il gatto' (bigramma 'topo mangia' assente); ignota: 'il pesce dorme'.
Bigrammi, corpus di 6 frasi, vocabolario senza soglia (unk_cutoff=1).

Perplexity ed entropia

Python

```
from nltk.util import ngrams
from nltk.lm.preprocessing import pad_both_ends
def test_ngrams(frase, n):
    return list(ngrams(pad_both_ends(frase, n), n))
t = test_ngrams('il gatto dorme'.split(), 2)
lm.perplexity(t) # 2.21 (MLE); lm.entropy(t) = log2(PP)
```

Parole sconosciute: Vocabulary e <UNK>

Python

```
from itertools import chain, takewhile
from nltk.lm import Vocabulary, KneserNeyInterpolated
padded = list(chain.from_iterable(
    pad_both_ends(s, 2) for s in sents))
voc = Vocabulary(padded, unk_cutoff=2) # rare -> <UNK>
lm_kn = KneserNeyInterpolated(2, vocabulary=voc)
lm_kn.fit(train) # train rigenerato; PP('il pesce dorme') = 14.3
```

Generazione di testo

Python

```
def genera(lm, seed=None):
    p = lm.generate(20, text_seed=['<s>'], random_seed=seed)
    return ' '.join(takewhile(lambda w: w != '</s>', p))
genera(lm, seed=1) # 'il topo dorme'
genera(lm, seed=3) # 'il gatto mangia la carne'
```

WordNet (1): synset e relazioni semantiche

nlTK.corpus.wordnet — database lessicale organizzato per significati

Synset, lemmi e definizioni

Python

```
from nltk.corpus import wordnet as wn

wn.synsets('bank') # tutti i sensi (Synset)
wn.synsets('bank', pos=wn.VERB) # solo i sensi verbali
s = wn.synset('bank.n.01') # formato lemma.pos.NN
s.definition() # glossa
s.examples() # frasi d'esempio
s.lemma_names() # sinonimi del synset
s.lemmas() # oggetti Lemma
s.pos(), s.name() # ('n', 'bank.n.01')
```

Output (indicativo)

```
'sloping land (especially the slope beside a body of water)'
['they pulled the canoe up on the bank',
 'he sat on the bank of the river and watched the currents']
```

Il nome ha forma lemma.pos.NN. POS: n (nome), v (verbo),
a (aggettivo), r (avverbio), s (aggettivo satellite).

Lemmi e forme base

```
wn.lemmas('bank') # Lemma di ogni senso
wn.morph('geese') # forma base: 'goose'
len(list(wn.all_synsets('n'))) # 82115 synset nominali
```

Relazioni semantiche

Metodo	Relazione
hypernys()	iperonimi: 'è un' (dog -> canine)
hyponyms()	iponimi: tipi più specifici
part_meronyms()	parti (auto -> ruota, porta)
member_holonys()	gruppi di cui è membro
entailments()	verbi implicati (snore -> sleep)
antonyms()	contrari (metodo di Lemma)

Percorsi nella gerarchia

Python

```
dog = wn.synset('dog.n.01')
dog.hypernys()
dog.hyponyms()[:3]
dog.hypernym_paths()[0][:4] # dalla radice
dog.root_hypernys()
```

Output (indicativo)

```
[Synset('canine.n.02'), Synset('domestic_animal.n.01')]
[Synset('basenji.n.01'), Synset('corgi.n.01'), Synset('cur.n.01')]
[Synset('entity.n.01'), Synset('physical_entity.n.01'),
 Synset('object.n.01'), Synset('whole.n.02')]
[Synset('entity.n.01')]
```

WordNet (2): similarità, disambiguazione, multilingua

path · Wu-Palmer · Lesk · Open Multilingual WordNet

Similarità semantica

Python

```
dog = wn.synset('dog.n.01')
cat = wn.synset('cat.n.01')
dog.path_similarity(cat)          # 0.2
dog.wup_similarity(cat)           # 0.857
dog.lch_similarity(cat)           # 2.028
dog.lowest_common_hypernyms(cat)
# [Synset('carnivore.n.01')]

from nltk.corpus import wordnet_ic
ic = wordnet_ic.ic('ic-brown.dat') # download: wordnet_ic
dog.lin_similarity(cat, ic)         # anche res_, jcn_
```

Quale misura usare

Metodo	Idea
path_similarity	$1 / (1 + \text{cammino minimo})$
wup_similarity	profondità del comune iperonimo
lch_similarity	cammino minimo in scala log
lin_/res_/jcn_similarity	basate su information content

Le misure lavorano su coppie di synset, non di parole: per due parole si confrontano i loro sensi e si prende il massimo.

Disambiguazione: algoritmo di Lesk

Python

```
from nltk.wsd import lesk
from nltk import word_tokenize

ctx = word_tokenize('I went to the bank to deposit money')
s = lesk(ctx, 'bank', pos='n')
print(s, '-', s.definition())
```

Limiti

Lesk sceglie il synset la cui glossa condivide più parole con il contesto. È una baseline debole: il senso restituito può non essere quello atteso (alternative: modelli neurali).

WordNet multilingua (OMW)

Python (richiede omw-1.4)

```
wn.langs()                # lingue disponibili
wn.synsets('cane', lang='ita') # da lemma italiano
d = wn.synset('dog.n.01')
d.lemma_names('ita')       # ['cane', 'Canis_familiaris']
d.lemma_names('spa')       # lemmi in spagnolo
```

La copertura dell'italiano è inferiore a quella dell'inglese.

FrameNet (1): frame, elementi e unità lessicali

nlk.corpus.framenet — semantica dei frame (Fillmore, ICSI Berkeley)

Concetti chiave

Termine	Significato
Frame	scena o situazione con i suoi partecipanti
Frame Element (FE)	ruolo semantico nel frame (Buyer, Goods)
Core / Peripheral	FE centrali vs. accessori (Time, Place)
Lexical Unit (LU)	lemma.POS che evoca il frame (buy.v)
Frame relations	Inheritance, Using, Subframe, ...
Annotazioni	esempi (exemplar) e testi annotati

Idea

Parole diverse (buy, purchase) evocano lo stesso frame e ne condividono i ruoli: FrameNet descrive il significato per scene, non per sinonimia.

Installazione e accesso

Python

```
import nltk
nltk.download('framenet_v17') # FrameNet 1.7
from nltk.corpus import framenet as fn
```

Prestazioni

Il primo accesso carica molti file XML: può richiedere alcuni secondi. FrameNet in NLTK riguarda solo l'inglese.

Esplorare frame, FE e LU

Python (indicativo)

```
len(fn.frames()) # oltre 1200 frame
fn.frames(r'(?i)^commerce') # ricerca per regex
f = fn.frame('Commerce_buy') # per nome (o per ID)
f.ID, f.name
f.definition # descrizione della scena
sorted(f.FE) # nomi dei frame element
f.FE['Buyer'].coreType # 'Core'
f.FE['Buyer'].definition
sorted(f.lexUnit) # LU: 'buy.v', 'purchase.v', ...
print(f) # scheda riassuntiva
```

Tipi di FE

coreType vale 'Core', 'Core-Unexpressed', 'Peripheral' oppure 'Extra-Thematic'. I FE 'Core' sono i ruoli indispensabili.

Dalla parola al frame

Python (indicativo)

```
fn.frames_by_lemma(r'(?i)^buy\.v') # frame evocati da buy.v
fn.lus(r'(?i)^purchase\.') # lexical unit per regex
f.lexUnit['buy.v'] # LU con la sua definizione
fn.fes('Buyer') # 'Buyer' in tutti i frame
```



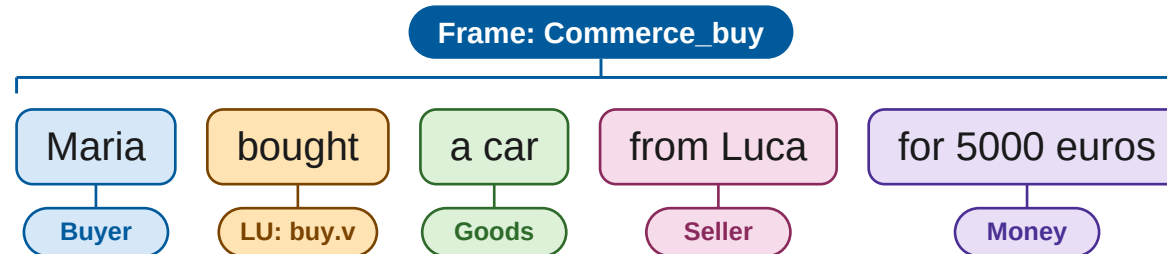
Università
degli Studi
di Palermo



FrameNet (2): frasi annotate e relazioni

esempio Commerce_buy · exemplar · relazioni tra frame · confronto con WordNet

Esempio: una frase, il frame evocato e i suoi elementi



La LU buy.v evoca il frame Commerce_buy; ogni sintagma della frase riempie un frame element (Buyer, Goods, Seller, Money).

Frase annotate e relazioni tra frame

Python (indicativo)

```
ex = fn.exemplars(r'buy\.v')    # frasi annotate a mano
print(ex[0])                    # testo + annotazioni
ex[0].text                       # solo il testo

fn.frame_relations('Commerce_buy')
fn.frame_relation_types()        # Inheritance, Using, ...
```

Le relazioni collegano i frame in una rete: Inheritance, Using, Subframe, Perspective_on, Precedes, Causative_of...

FrameNet e WordNet a confronto

	WordNet	FrameNet
Unità	synset (sinonimi)	frame (scena)
Relazioni	iperonimia, meronimia	Inheritance, Using
Ruoli semantici	assenti	frame element
Dimensione	circa 117.000 synset	oltre 1.200 frame
Lingue	molte (OMW)	inglese in NLTK

Attenzione

NLTK consente di consultare FrameNet, ma non di etichettare automaticamente frasi nuove (frame-semantic parsing): servono sistemi dedicati (es. Open-SESAME, LOME) o modelli transformer.

Riepilogo e risorse in rete

NLTK in una pagina

Argomenti trattati

- **Importazione:** `pip install nltk` · `import nltk` · moduli
- **Risorse:** `nltk.download(...)` · `nltk.data.path`
- **Frase e parole:** `sent_tokenize` · `word_tokenize`
- **Tokenizzatori:** `Regex` · `Tweet` · `MWE` · `Whitespace` · `span`
- **Stemming:** `Porter` · `Lancaster` · `Snowball` (italiano incl.)
- **Lemmatizzazione:** `WordNetLemmatizer` + `pos` ('n','v','a','r')
- **POS tagging:** `pos_tag` · `tagset Penn` / 'universal'
- **NER:** `ne_chunk` · `tree2conlltags` · `RegexParser`
- **Utilità:** `FreqDist` · `Text` · `ngrams` · `edit_distance` · metriche
- **Language model:** `nltk.lm`: `MLE`, `Laplace`, `KneserNey` · `perplexity`
- **WordNet:** `synset` · relazioni · similarità · `Lesk` · `OMW`
- **FrameNet:** `frame` · `frame element` · `lexical unit` · relazioni
- **Lessici:** `stopwords` · `VADER`

Alternative e strumenti complementari

`spaCy` — pipeline industriali, modelli per l'italiano
`Stanza` — tagger, lemmatizzatore, NER multilingua
`transformers` — modelli pre-addestrati Hugging Face
`gensim` — word embedding e topic modeling
`simplemma` — lemmatizzazione leggera multilingua

Risorse su Internet

- **Sito e documentazione:** <https://www.nltk.org/>
- **Libro NLTK (gratuito):** <https://www.nltk.org/book/>
- **Riferimento API:** <https://www.nltk.org/api/nltk.html>
- **Installare i dati:** <https://www.nltk.org/data.html>
- **HOWTO (indice):** <https://www.nltk.org/howto.html>
- **Modulo nltk.lm:** <https://www.nltk.org/api/nltk.lm.html>
- **HOWTO WordNet:** <https://www.nltk.org/howto/wordnet.html>
- **HOWTO FrameNet:** <https://www.nltk.org/howto/framenet.html>
- **FrameNet (ICSI):** <https://framenet.icsi.berkeley.edu/>
- **Codice sorgente:** <https://github.com/nltk/nltk>
- **Dati (nltk_data):** https://github.com/nltk/nltk_data
- **WordNet:** <https://wordnet.princeton.edu/>
- **Snowball stemmer:** <https://snowballstem.org/>
- **Universal Dependencies:** <https://universaldependencies.org/u/pos/>